

Artix-7 FPGA Board Documentation

This document provides a step-by-step guide for performing a basic experiment—e.g.: blinking an onboard LED—using the new Artix-7 based FPGA boards designed at WEL, IIT Bombay.

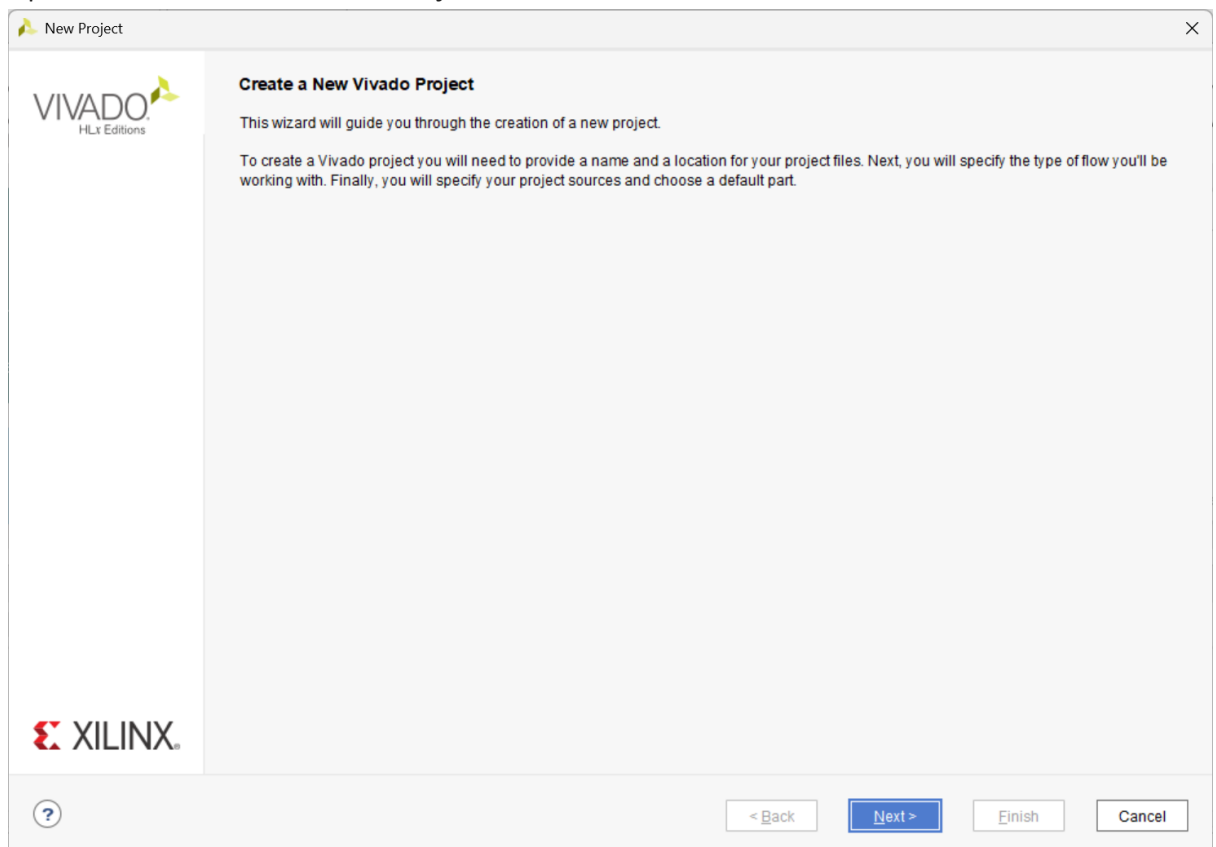
1. Blinking Onboard LED using VHDL

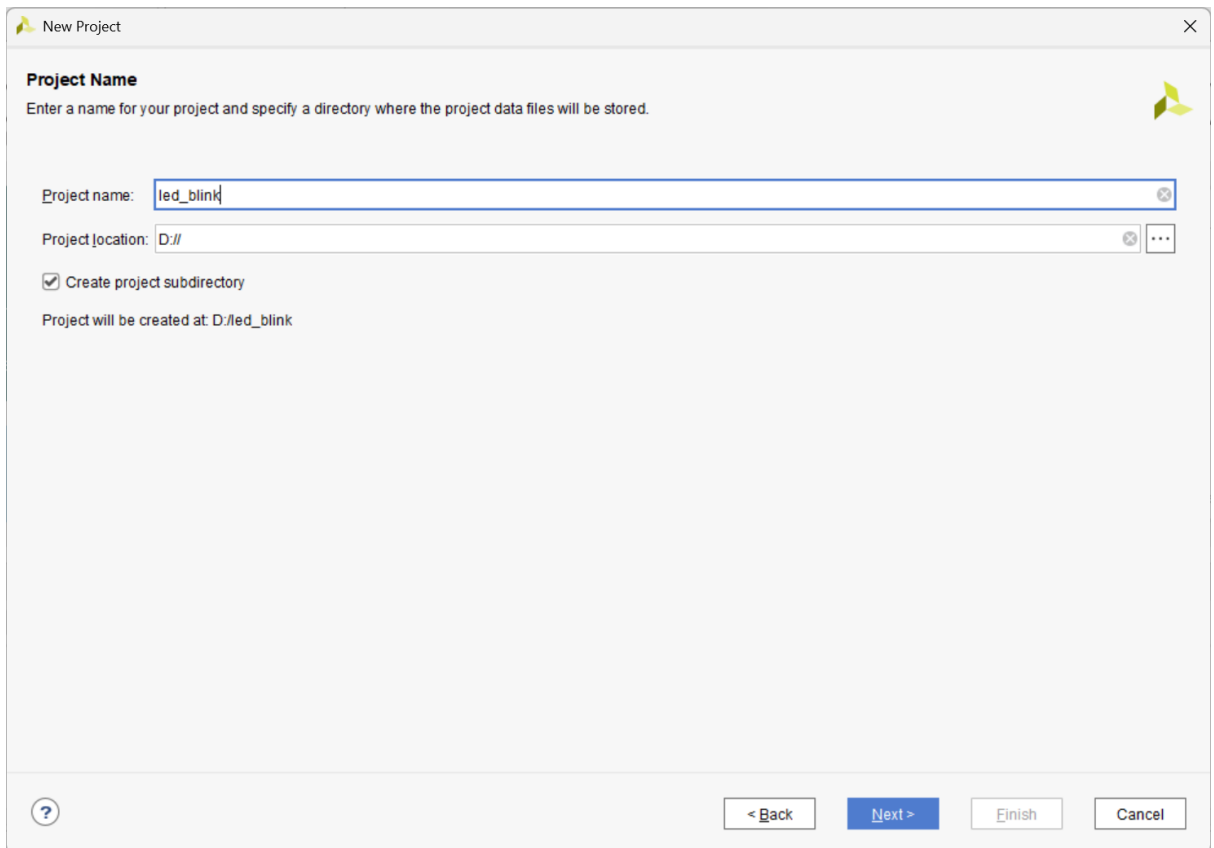
This method involves writing a Hardware Description Language (HDL) code (VHDL in this case) to directly control the LED via the FPGA fabric, and then using the Vivado Design Suite to synthesize, implement, and generate the bitstream.

Step-by-Step Guide in Vivado

Step 1: Create a New Vivado Project

1. Open Vivado and select **"Create Project."**





New Project

Project Name
Enter a name for your project and specify a directory where the project data files will be stored.

Project name:

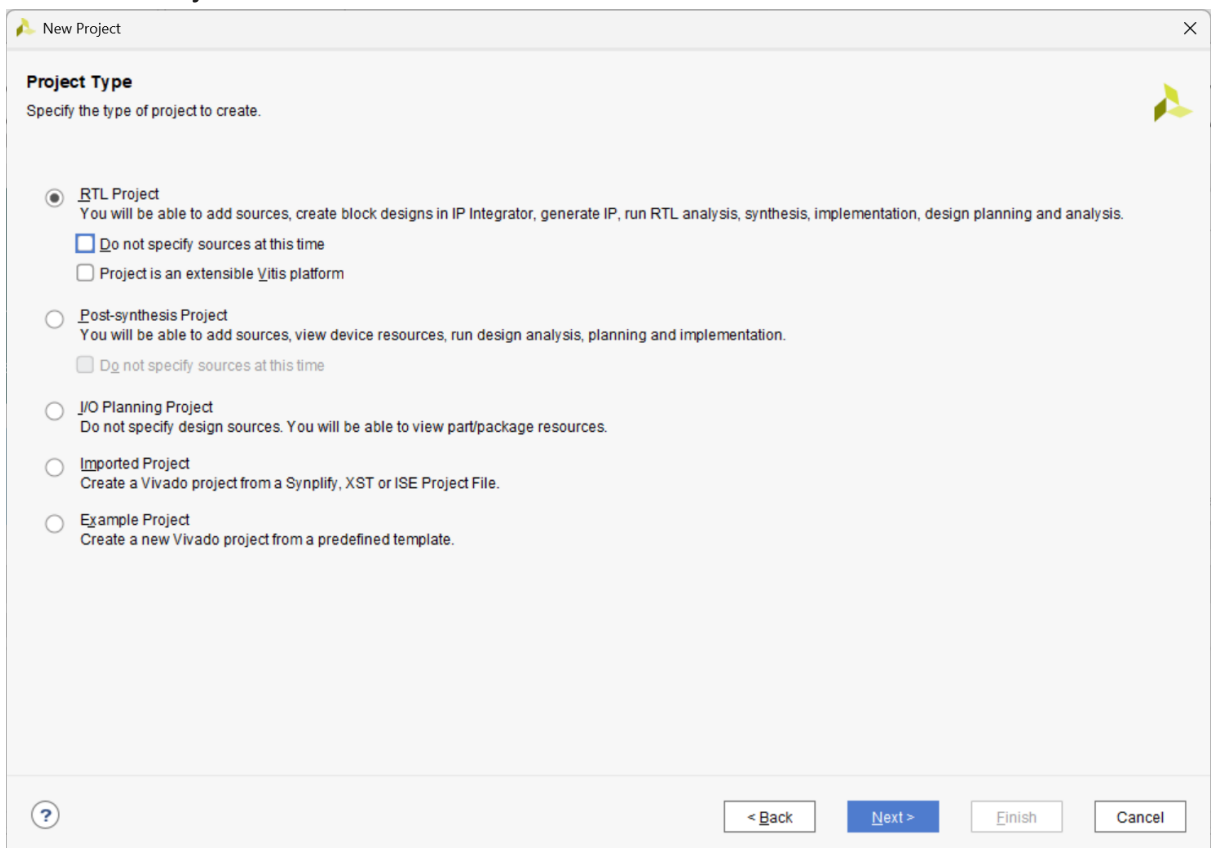
Project location:

☒ Create project subdirectory

Project will be created at: D:/led_blink

[?<](#) [Next >](#) [Finish](#) [Cancel](#)

2. Choose **RTL Project** and click **Next**.



New Project

Project Type
Specify the type of project to create.

☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.
☐ Do not specify sources at this time
☐ Project is an extensible Vitis platform

☐ **Post-synthesis Project**
You will be able to add sources, view device resources, run design analysis, planning and implementation.
☐ Do not specify sources at this time

☐ **I/O Planning Project**
Do not specify design sources. You will be able to view part/package resources.

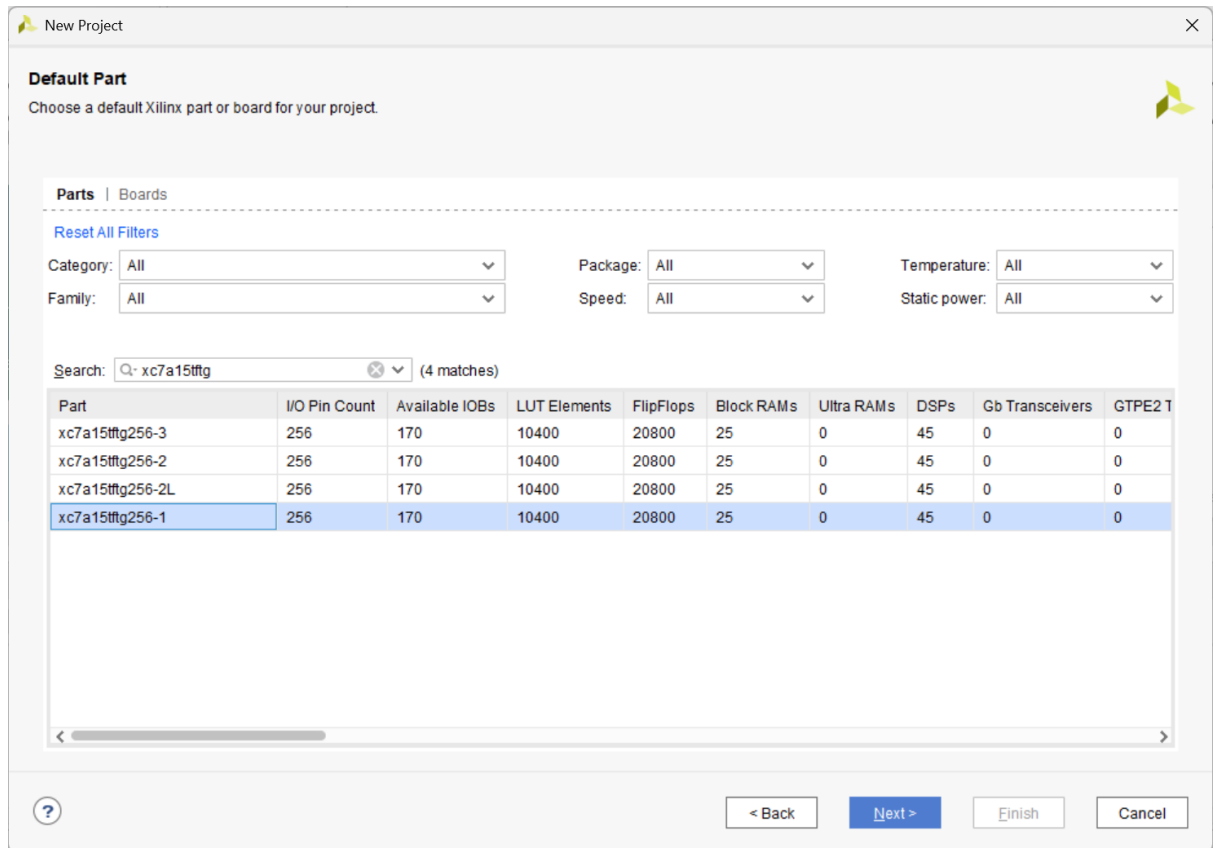
☐ **Imported Project**
Create a Vivado project from a Synplify, XST or ISE Project File.

☐ **Example Project**
Create a new Vivado project from a predefined template.

[?<](#) [Next >](#) [Finish](#) [Cancel](#)

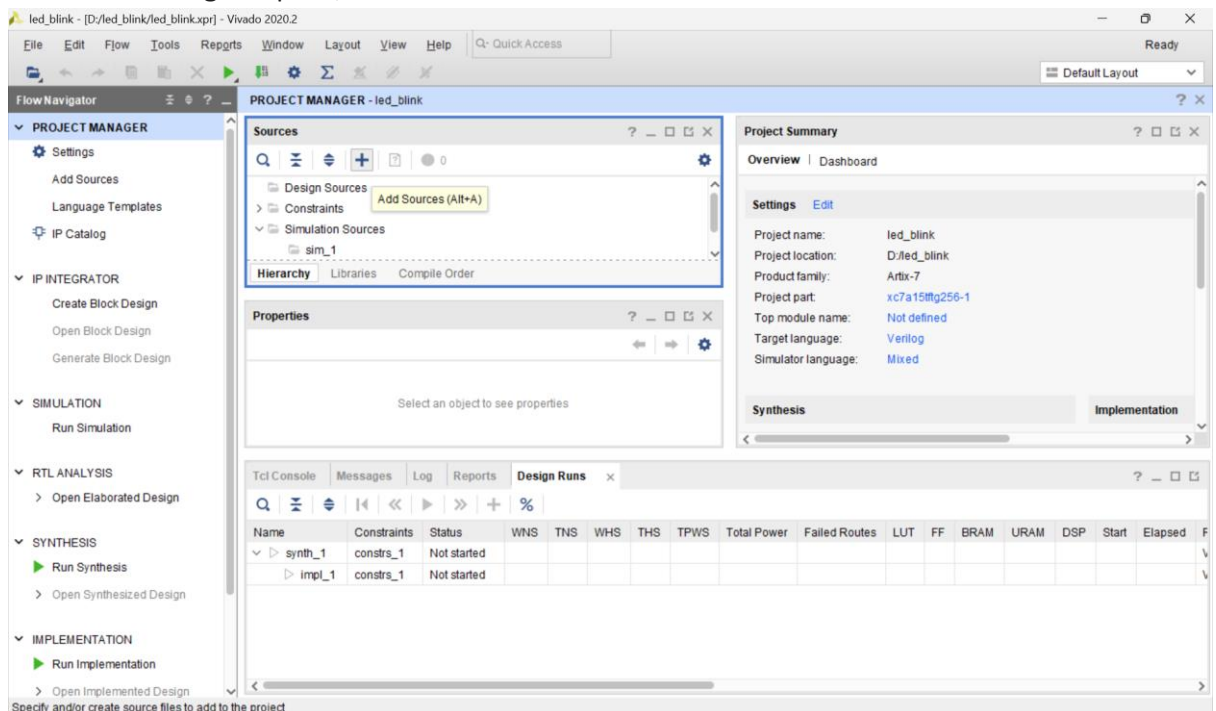
3. Specify the project name (e.g., Artix7_LED_Blink_VHDL) and project location.
4. Add design sources later; click **Next**.

5. Add constraints later; click **Next**.
6. Select the specific Artix-7-part number “**XC7A15TFTG256-1**”. If a custom board file is available, select it here. Click **Next** and then **Finish**.

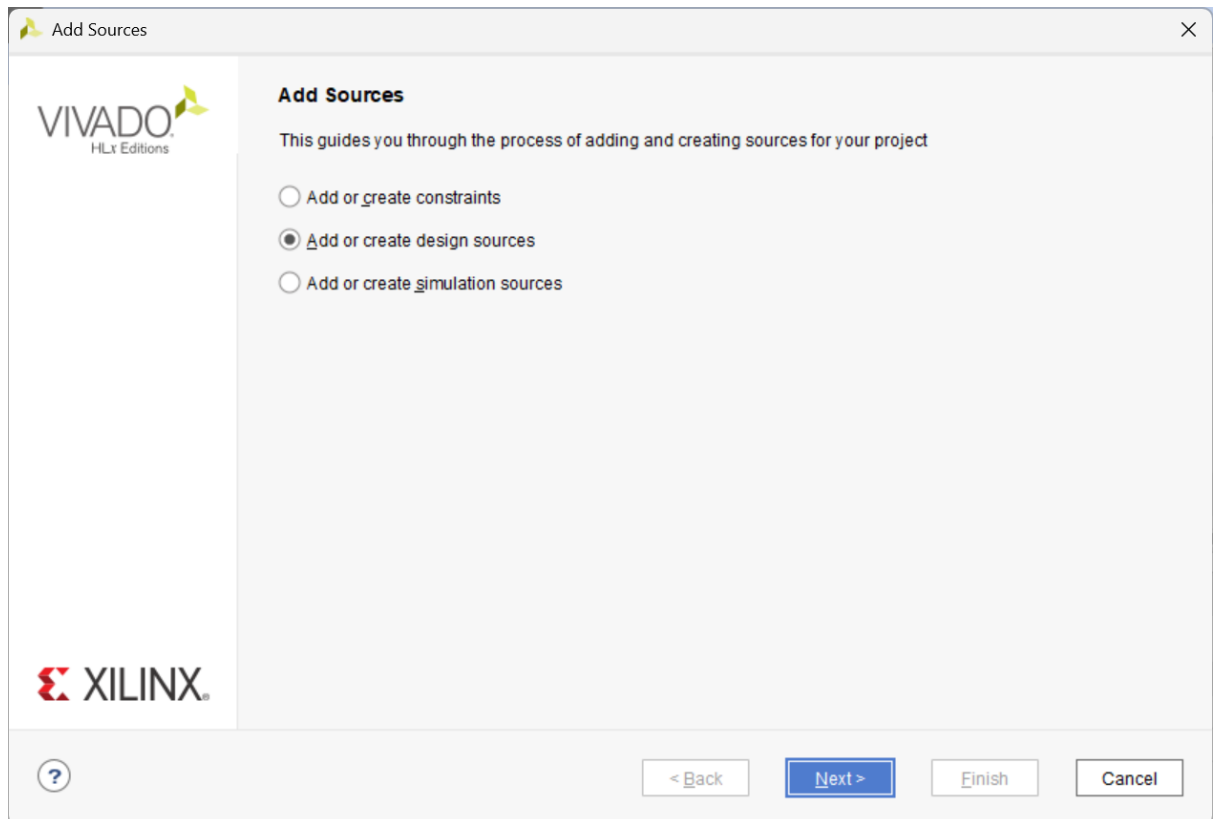


Step 2: Create VHDL Source File

1. In the **Flow Navigator** pane, select **Add Sources**.



2. Choose **Add or Create Design Sources** and click **Next**.



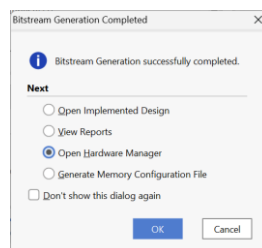
3. Click **Create File**, name the file (e.g., led_blinker.vhd), and set the File type to **VHDL**. Click **OK** and then **Finish**.
4. Vivado will prompt to define the module ports. For the onboard LED, you will typically need:
 - clk (Input).
 - reset (Input).
 - led (Output).
5. Click **OK**.

Step 3: Add Constraints (Pin Mapping)

1. In the **Flow Navigator**, select **Add Sources** -> **Add or Create Constraints**.
2. Click **Create File**, name it (e.g., artix7_pins.xdc), and click **OK** and then **Finish**.
3. Open the newly created XDC file and add the pin assignments based on the board mappings.

Step 5: Implement and Generate Bitstream

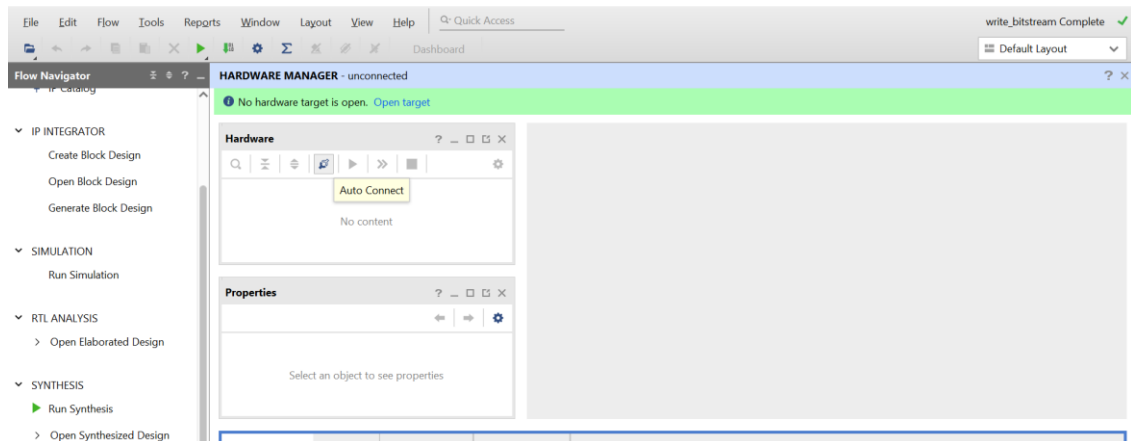
1. In the **Flow Navigator**, click **Run Synthesis**.
2. Once Synthesis is complete, click **Run Implementation**.
3. After Implementation, click **Generate Bitstream**.



Step 6: Program the FPGA

1. Connect the Artix-7 board to the PC using the JTAG programmer (usually onboard).
2. In Vivado, go to **Program and Debug** -> **Open Hardware Manager**.

3. Click **Open Target** -> **Auto Connect**. Vivado should detect the FPGA device.



4. Select the device and click **Program Device**.
5. Browse to the generated bitstream file (e.g., Artix7_LED_Blink_VHDL.runs/impl_1/led_blinker.bit) and click **Program**.

The onboard LED should now start blinking at a desired rate (maybe using clock divider).

2. Blinking Onboard LED using MicroBlaze Softcore Processor

This method uses a MicroBlaze soft-core processor. Vivado is used to build the hardware platform (MicroBlaze + GPIO), and Vitis is used to run C code that sends commands to blink the LED.

Step-by-Step Guide in Vivado (Hardware Platform)

Step 1: Create a New Vivado Project

1. Open Vivado and select "Create Project."
2. Choose RTL Project and select "Do not specify sources at this time."
3. Select the specific Artix-7 part number XC7A15TFTG256. Click Finish.

Step 2: Create a Block Design

1. In the Flow Navigator, select Create Block Design.
2. Name it (e.g., system) and click OK.

Step 3: Add and Configure IPs

1. In the block design canvas, click the Add IP button.
2. Add the MicroBlaze processor.
3. Add the AXI GPIO IP (for controlling the onboard LED).

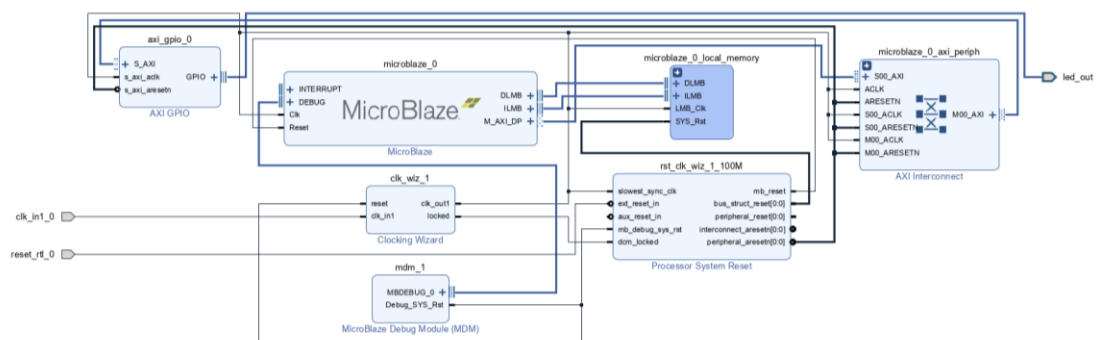
Step 4: Run Automation and Configure Clocks/Resets

1. Click Run Block Automation for the MicroBlaze (default settings are usually fine for simple designs).
2. Click Run Connection Automation to automatically connect the MicroBlaze and peripherals using AXI interconnects.

3. CRITICAL: Double-click the generated Clocking Wizard IP. Ensure the primary input clock is set to 100 MHz.
4. Scroll to the bottom of the Clocking Wizard settings and set the Reset Type to Active High (matching the physical push-button on pin F4).
5. Ensure the "Processor System Reset" IP is also configured for Active High.

Step 5: Configure AXI GPIO and Make External Ports

1. Double-click the AXI GPIO IP.
2. Set All Inputs to false and All Outputs to true (since we are driving the LED).
3. Set the Width of GPIO to 1 (for a single LED). Click OK.
4. Expand the GPIO interface by clicking the + icon on the IP block. Right-click the gpio_io_o port and select Make External.
5. Rename this new external port to led.
6. Right-click the primary clock input on the Clocking Wizard and select Make External. Rename it to clk.
7. Right-click the primary reset input and select Make External. Rename it to reset_0.



8. Final Block Design showing external ports named clk, reset_0, and led

Step 6: Add Constraints and Generate Bitstream

1. In the Sources pane, right-click the block design (system.bd) and select Create HDL Wrapper (let Vivado manage wrapper).
2. In the Flow Navigator, select Add Sources -> Add or Create Constraints. Create a new file named pins.xdc.
3. Paste the following verified constraints into the XDC file:

```
Tcl
# System Clock (100 MHz)
set_property PACKAGE_PIN N14 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add
[get_ports clk]

# System Reset (Active High)
set_property PACKAGE_PIN F4 [get_ports reset_0]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports reset_0]
```

```
# User LED Output
```

```
set_property PACKAGE_PIN C11 [get_ports led]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports led]
```

```
set_property DRIVE 8 [get_ports led]
```

```
set_property SLEW SLOW [get_ports led]
```

4. Click Generate Bitstream (this will run Synthesis and Implementation automatically).

Step 7: Export Hardware to Vitis

1. Once the Bitstream is generated, go to File -> Export -> Export Hardware.
2. Select Include Bitstream and export to a designated directory. This creates the XSA (Xilinx Shell Archive) file.

Step-by-Step Guide in Vitis (Software Development)

Step 1: Launch Vitis & Configure Platform

1. Open Vitis and set a workspace location.
2. Select "Create Application Project".
3. Platform Selection: Click Create a new platform from hardware (XSA). Browse to the XSA file exported from Vivado. Click Next.
4. [Screenshot Placeholder: Vitis platform selection screen showing the imported XSA]

Step 2: Configure Application Project

1. Give the application project a name (e.g., blink_vitis). Select the MicroBlaze processor. Click Next.
2. Keep the default Domain settings (Standalone, C/C++).
3. Select the "Empty Application" template. Click Finish.
4. [Screenshot Placeholder: Vitis Template Selection screen showing Empty Application]

Step 3: Write the Application Code

1. In the Explorer pane, right-click the src folder inside your application project -> New -> File. Name it main.c.
2. Paste the following C code to control the GPIO:

```
C
#include "xparameters.h"
#include "xgpio.h"
#include "sleep.h"
#include "xil_printf.h"

int main() {
    XGpio Gpio;
    int Status;

    xil_printf("Starting LED Blink Experiment...\r\n");

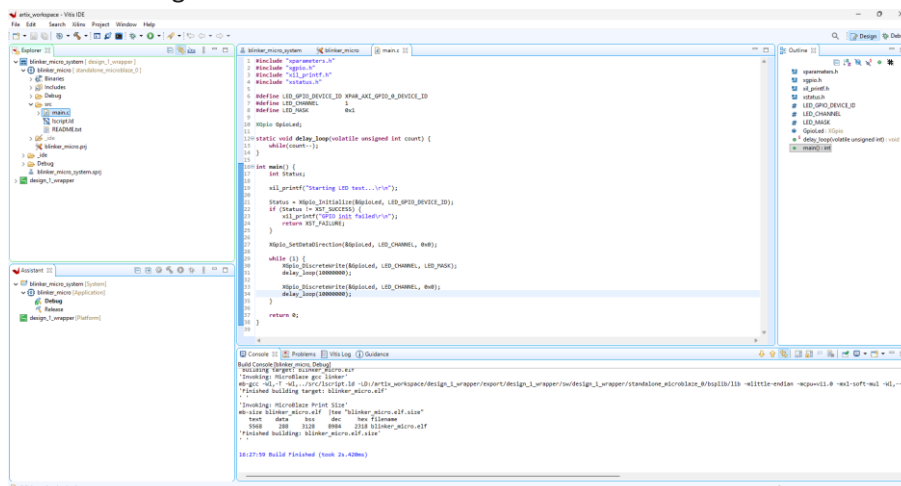
    // Initialize the GPIO driver
    Status = XGpio_Initialize(&Gpio, XPAR_AXI_GPIO_0_DEVICE_ID);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }
}
```

```
// Set the direction for channel 1 to output (0 = Output)
XGpio_SetDataDirection(&Gpio, 1, 0x0);
```

```
while (1) {
    XGpio_DiscreteWrite(&Gpio, 1, 0x1); // Turn on LED
    usleep(500000);                      // Wait 500ms
    XGpio_DiscreteWrite(&Gpio, 1, 0x0); // Turn off LED
    usleep(500000);                      // Wait 500ms
}
return 0;
}
```

Step 4: Compile and Run

1. Right-click the application project (blink_vitis) and select Build Project.
2. Ensure the Artix-7 board is connected via the JTAG programmer and powered on.
3. Right-click the application project and select Run As -> Launch on Hardware (Single Application Debug).
4. The Vitis software will automatically program the hardware bitstream onto the FPGA, download the compiled executable (.elf file) to the MicroBlaze processor, and the onboard LED will start blinking under software control.



Vitis IDE showing successful run console output and active debug session

3.Displaying Text on 16x2 LCD using VHDL

Step-by-Step Guide in Vivado

Step 1: Create a New Vivado Project

1. Open Vivado and select Create Project.
2. Choose RTL Project and click Next.
3. Specify project name (for example: Artix7_LCD_VHDL) and project location.
4. Choose to add design sources later and click Next.
5. Choose to add constraints later and click Next.
6. Select part number XC7A15TFTG256-1. Click Next and then Finish.

Step 2: Create VHDL Source File

1. In Flow Navigator, select Add Sources.
2. Choose Add or Create Design Sources and click Next.

3. Click Create File, name it (for example: lcd_controller.vhd), set file type to VHDL, then click OK and Finish.
4. Define module ports for LCD interface and system signals.
5. Ensure ports include clock, reset, LCD data bus, and LCD control signals.

Step 3: Add Constraints (Pin Mapping)

1. In Flow Navigator, select Add Sources and then Add or Create Constraints.
2. Create a new XDC file (for example: lcd_pins.xdc).
3. Add verified board pin mappings for all LCD signals and required system signals.
4. Save constraints and confirm there are no unconstrained top-level ports.

Step 4: Verify LCD Hardware Connections

1. Confirm LCD operates in write-only mode for this experiment.
2. Ensure RS and E are connected to FPGA control outputs.
3. Ensure data lines D0 to D7 are connected to FPGA data outputs.
4. Tie LCD RW permanently to GND.
5. Connect LCD VSS to GND and VDD to 5V.
6. Connect LCD contrast pin through potentiometer and adjust for clear visibility.
7. Connect backlight pins as per board/LCD module requirement.

Step 5: Implement and Generate Bitstream

1. Click Run Synthesis.
2. After synthesis completes, click Run Implementation.
3. After implementation completes, click Generate Bitstream.

Step 6: Program the FPGA

1. Connect the Artix-7 board to the PC using JTAG and power it on.
2. In Vivado, go to Program and Debug and open Hardware Manager.
3. Click Open Target and select Auto Connect.
4. Select detected FPGA device and click Program Device.
5. Select generated bitstream and program the device.

Step 7: Verify LCD Output

1. Adjust LCD contrast slowly until characters are visible.
2. Confirm text appears on line 1 and line 2 as expected.
3. If only black blocks appear, check initialization sequencing and control/data wiring.
4. Displaying Text on 16x2 LCD using MicroBlaze Softcore Processor
This method uses MicroBlaze with AXI GPIO. Vivado creates the hardware platform, and Vitis runs a C application that initializes the LCD and writes text.

Step-by-Step Guide in Vivado (Hardware Platform)

Step 1: Create a New Vivado Project

1. Open Vivado and select Create Project.
2. Choose RTL Project and select Do not specify sources at this time.
3. Select part number XC7A15TFTG256-1 and click Finish.

Step 2: Create a Block Design

1. In Flow Navigator, select Create Block Design.
2. Name it (for example: design_1) and click OK.

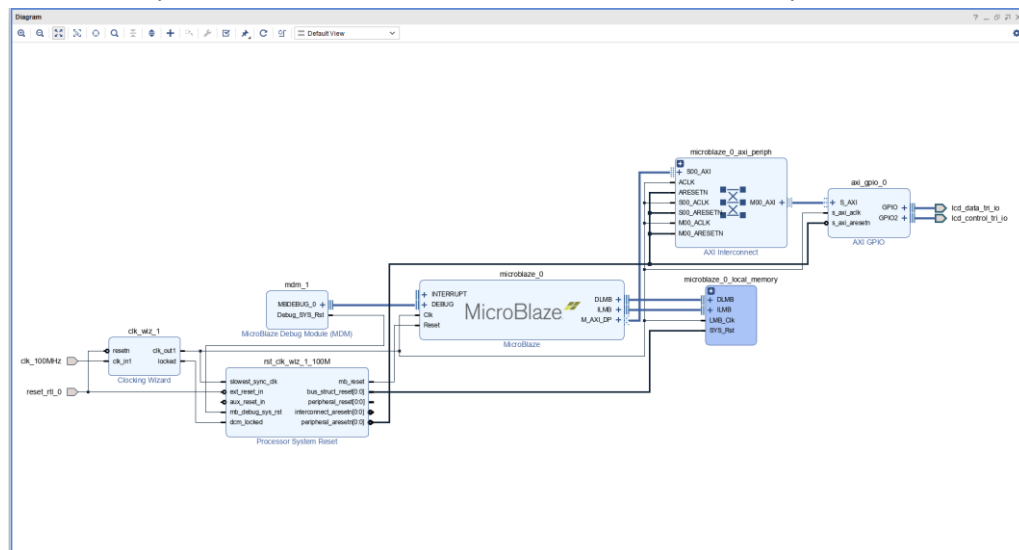
Step 3: Add and Configure IP

1. Add MicroBlaze IP.
2. Add AXI GPIO IP for LCD interface.

3. Enable Dual Channel in AXI GPIO.
4. Configure Channel 1 width as 8 bits for LCD data bus.
5. Configure Channel 2 width as 2 bits for LCD control lines RS and E.
6. Set both channels as outputs.
7. Add clocking and reset infrastructure if not auto-inserted.

Step 4: Run Automation

1. Run Block Automation for MicroBlaze.
2. Run Connection Automation for AXI and peripheral connectivity.
3. Validate block design and resolve any warnings/errors.
4. Ensure external ports are created for LCD data bus, LCD control bus, system clock, and reset.



Step 5: Create HDL Wrapper

1. In Sources pane, right-click block design and select Create HDL Wrapper.
2. Choose Let Vivado manage wrapper and continue.

Step 6: Add Constraints and Pin Assignments

1. Create a constraints file (for example: lcd.xdc).
2. Assign verified board pins for LCD signals using your approved board mapping.
3. Recommended LCD pin mapping used in this experiment:
4. RS to M16
5. E to N16
6. D0 to P15
7. D1 to P16
8. D2 to R15
9. D3 to R16
10. D4 to T14
11. D5 to T15
12. D6 to P14
13. D7 to R13
14. Keep RW tied to GND on hardware.
15. Add system clock and reset constraints according to your board clock/reset wiring.

Step 7: Generate Bitstream

1. Click Generate Bitstream.

2. Vivado runs synthesis and implementation automatically.
3. Confirm successful bitstream generation.

Step 8: Export Hardware to Vitis

1. Go to File and Export and Export Hardware.
2. Enable Include Bitstream.
3. Export and save the XSA file.

Step-by-Step Guide in Vitis (Software Development)

Step 1: Launch Vitis and Create Application

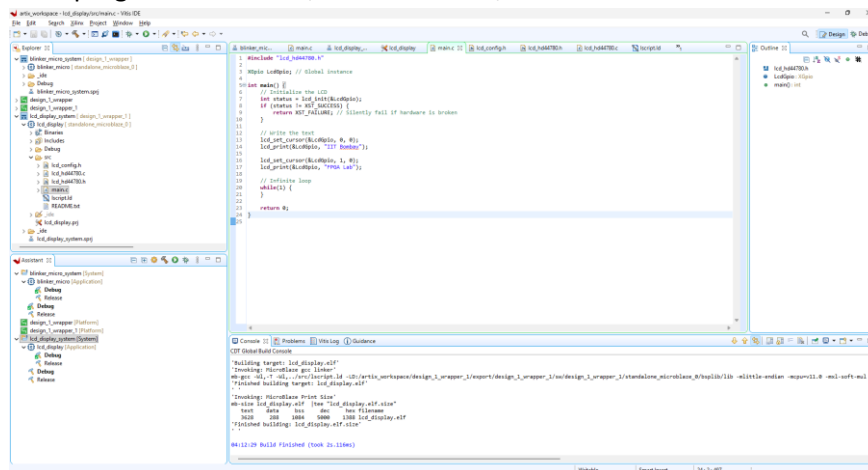
1. Open Vitis and select workspace.
2. Create Application Project.
3. Create platform from the exported XSA file.
4. Select MicroBlaze processor target and continue.
5. Choose Empty Application template and finish.

Step 2: Implement LCD Application Logic

1. Initialize AXI GPIO driver for the configured GPIO IP.
2. Set GPIO channel directions to output.
3. Implement HD44780 write-only command/data flow using RS and E control lines.
4. Use proper LCD startup sequence before sending text.
5. Write text to line 1 and line 2 DDRAM addresses.

Step 3: Build and Run

1. Build application project.
2. Connect board through JTAG and power it on.
3. Launch on Hardware.
4. Confirm Vitis programs bitstream, downloads ELF, and starts execution.



Step 4: Validate on LCD

1. Confirm initialization clears black-block default state.
2. Confirm readable characters appear on both lines.
3. If display is blank or only blocks are visible, verify contrast and RW grounding first, then verify RS/E and data pin continuity.

4. Potentiometer to Onboard LEDs using XADC

Hardware Setup (Analog Input Wiring)

Use your board schematic to find which analog header pins go to which XADC channel. General safe wiring for a potentiometer:

1. One end of potentiometer -> 3.3V
2. Other end -> GND
3. Middle pin (wiper) -> selected XADC analog input

Important:

- XADC analog inputs are special-purpose pins. Always follow your board schematic.
- Ensure the input voltage does not exceed the allowed range (depends on your board's scaling/divider network).

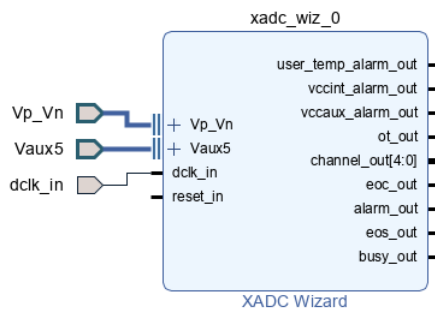
Step-by-Step Guide in Vivado

Step 1: Create or Open the Vivado Project

1. Open Vivado.
2. Open the target project for XADC experiment.
3. Ensure target part is XC7A15TFTG256-1.
4. Confirm project clock source is 100 MHz input for XADC DCLK.

Step 2: Add the Top Design for XADC-to-LED Logic

1. Add the RTL source for the new top module dedicated to this experiment.
2. Set this module as Top in project sources.
3. Ensure top-level ports include:
4. dclk_in
5. Vaux5_v_p
6. Vaux5_v_n
7. led[7:0]
8. Save and re-check hierarchy for clean elaboration.



Re-customize IP

XADC Wizard (3.3)

Documentation IP Location

☐ Show disabled ports

Component Name: xadc_wiz_0

Basic | ADC Setup | Alarms | Channel Sequencer | Summary

Interface Options

☐ AXI4-Lite ☐ DRP ☒ None

Startup Channel Selection

☐ Simultaneous Selection
☐ Independent ADC
☐ Single Channel
☒ Channel Sequencer

ADASTREAM Options

☐ Enable ADASTREAM
 FFO Depth: 7 (0 - 1023)

Control/Status Ports

☒ reset_in ☐ Temp Bus ☐ JTAG Arbiter
 Event Mode Trigger: ☒ convst_in ☐ convstn_in

Timing Mode

☒ Continuous Mode ☐ Event Mode

DRP Timing Options

☒ Enable DRP
 DRP Frequency (MHz): 100 (0.0 - 250.0)
 ADC Conversion Rate (SPS): 1000 (0.0 - 1000.0)
 Acquisition Time (CLK): 4
 Clock divider value = 4
 ADC Clock Frequency (MHz) = 25.00

Analog Stimulus File Options

SimFile Selection: Default
 Analog Stimulus File: design
 SimFile Location: /
 Waveform Type: CONSTANT
 Frequency (kHz): 1.0 (0.1 - 100.0)
 Number of Traces: 1 (1 - 1000)

OK Cancel

Re-customize IP

XADC Wizard (3.3)

Documentation IP Location

☐ Show disabled ports

Component Name: xadc_wiz_0

Basic | **ADC Setup** | Alarms | Channel Sequencer | Summary

Sequencer Mode: Continuous Channel Averaging: None

ADC Calibration

☐ ADC Offset Calibration ☐ Sensor Offset Calibration
☒ ADC Offset and Gain Calibration ☒ Sensor Offset and Gain Calibration

☒ Enable CALIBRATION Averaging

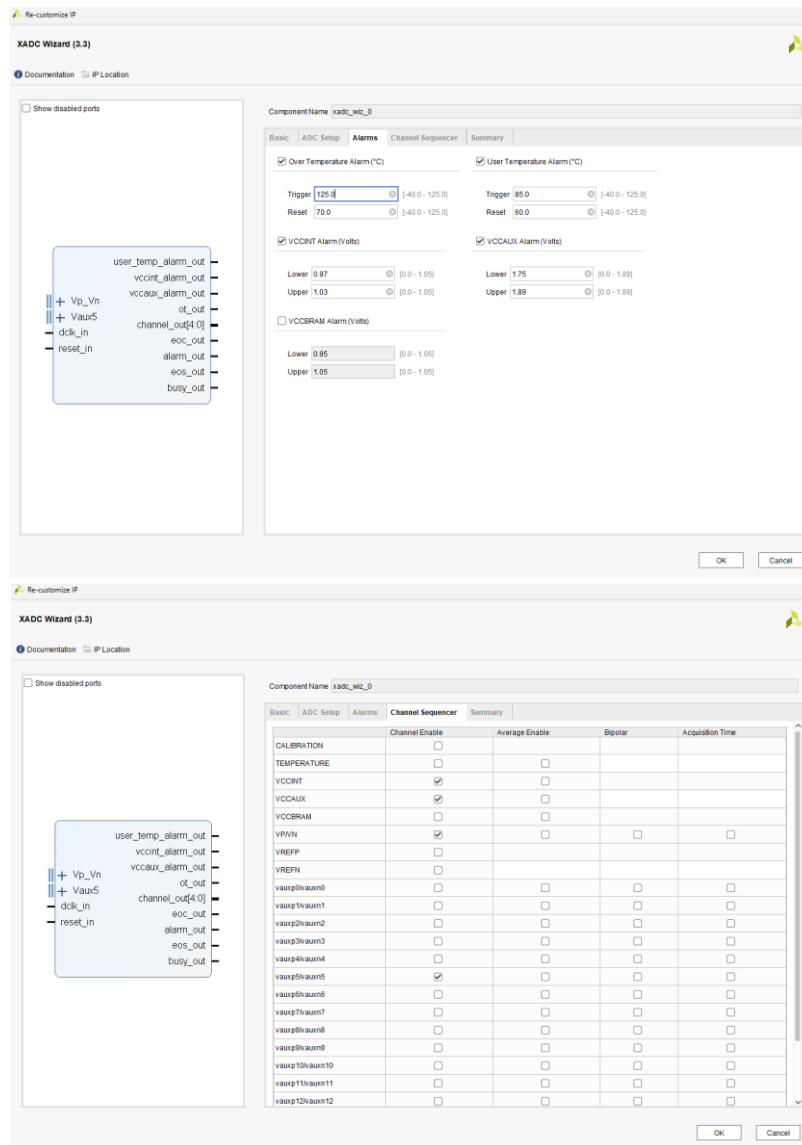
External Multiplexer Setup

☐ External Multiplexer
 Channel for MUX: VP_VN
☐ Enable muxa0b_out port

Power Down Options

☐ ADCB
☒ ADCA

OK Cancel



Step 3: Configure Pin Constraints

1. Open constraints file and map system clock input.
2. Map potentiometer differential analog pair to VAUX5 pins.
3. Map 8 onboard LEDs to LED bus bits in LSB-to-MSB order.
4. Apply I/O standard and output drive/slew for LED pins.
5. Save constraints and verify there are no unconstrained required ports.

Step 4: Build the Hardware

1. Run Synthesis.
2. Run Implementation.
3. Generate Bitstream.
4. Confirm no critical errors in DRC, timing, or routing reports.

Step 5: Program the FPGA

1. Connect board power and JTAG.
2. Open Hardware Manager.
3. Open Target and Auto Connect.
4. Select detected FPGA.
5. Program using the generated bitstream for this experiment.

Step 6: Perform the Potentiometer Test

1. Rotate potentiometer slowly from minimum to maximum.
2. Observe onboard LEDs as a binary count pattern.
3. Verify increasing voltage causes increasing binary LED value.
4. Verify decreasing voltage causes decreasing binary LED value.

Step 7: Validation Criteria

1. LEDs respond continuously to potentiometer movement.
2. Binary transitions are stable and monotonic over most of travel.
3. No fixed stuck pattern across full potentiometer sweep.

4. SPI Flash configuration for standalone boot

Goal: program an external SPI flash so the board boots automatically after power-up (no JTAG needed).

Important: standalone boot requires correct BOOT MODE straps/jumpers on your custom board. Keep your board's boot-mode table handy.

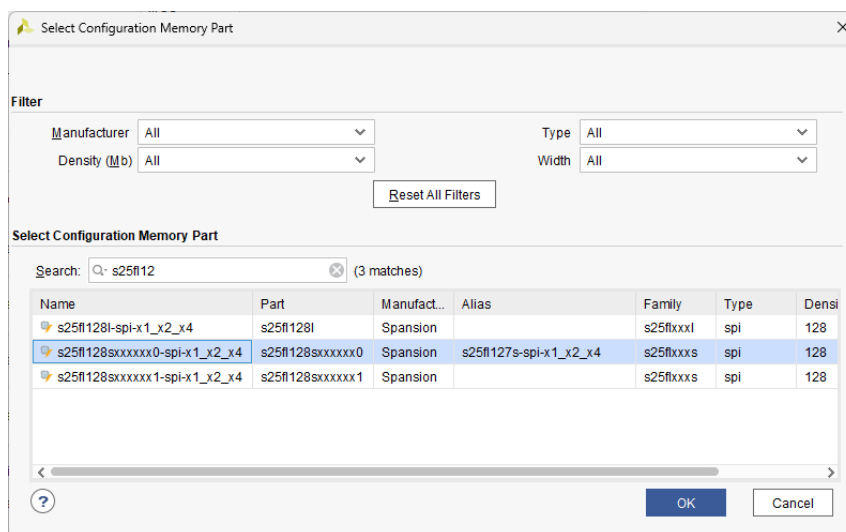
Generate configuration memory file (.mcs)

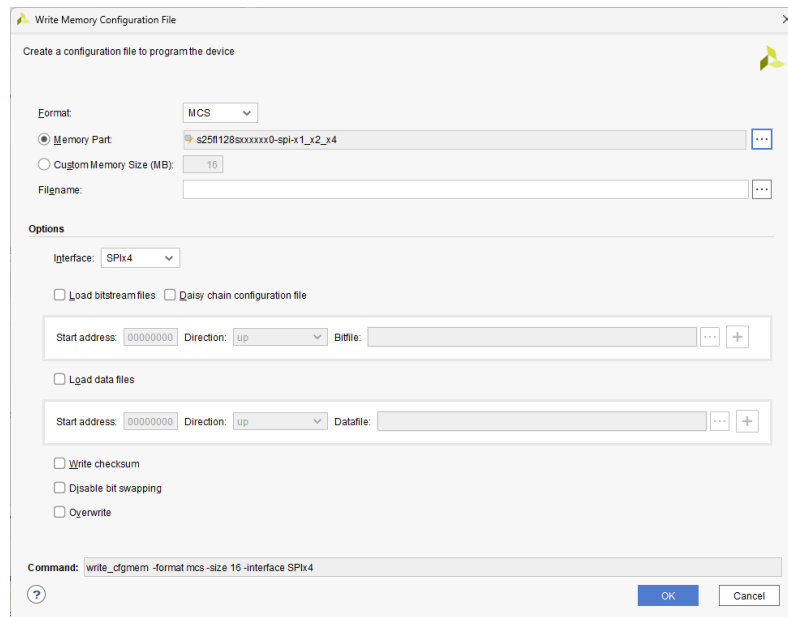
1. After generating a bitstream (.bit), create a flash image file (.mcs) using Vivado.
2. Open the Vivado project for the design you want to boot (VHDL or MicroBlaze).
3. Ensure 'Generate Bitstream' completed successfully.
4. Go to Tools -> Generate Memory Configuration File.

Select:

- Format: MCS
- Memory part: choose your SPI flash model (edit based on your flash)
- Interface: SPIx1 or SPIx4 depending on your flash wiring and design (often SPIx1 for configuration)
- Load bitstream file: select your .bit
- (MicroBlaze designs) ensure the bitstream already includes BRAM init from ELF (Vivado does this when you associate ELF before bitgen).

5. Generate the .mcs file and note the output path.



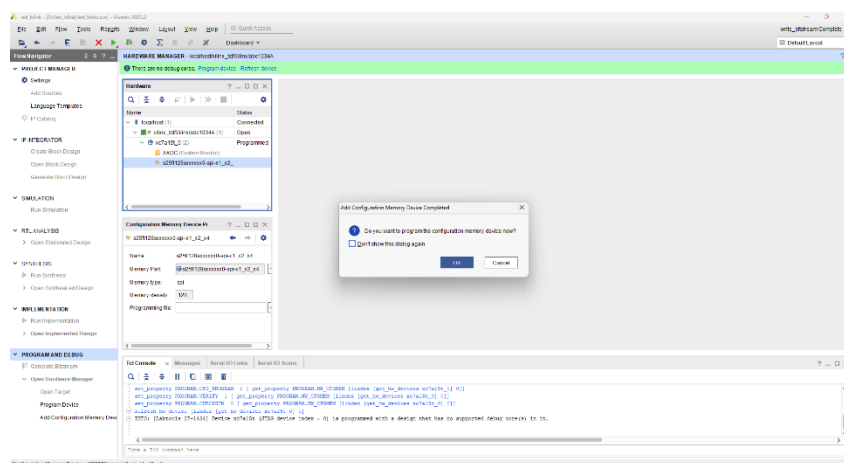


Generate Memory Configuration File wizard

Show format=MCS and selected configuration memory part.

9.3 Program the SPI flash using Hardware Manager

1. Connect board via JTAG and power it ON.
2. Vivado -> Open Hardware Manager -> Open Target -> Auto Connect.
3. Right-click the FPGA device -> Add Configuration Memory Device.
4. Select your flash model from the list (must match your board).
5. Vivado may prompt for a .bit file to use for temporary programming over JTAG; select your bitstream.
6. After config memory device appears under FPGA, right-click it -> Program Configuration Memory Device.
7. Select:
 - Configuration file: your generated .mcs
 - (Optional) Verify check-box enabled
8. Program.



Hardware Manager: Add Configuration Memory Device

Verification steps:

1. Power OFF the board.
2. Set boot mode jumpers to SPI flash boot (<EDIT> settings).
3. Power ON the board.

Expected result: design runs automatically (LED blinks / LCD shows something / UART prints if connected).

5.UART

Hardware Setup

1. Master board switches:
 1. SW0 -> F4
 2. SW1 -> G4
 3. SW2 -> H4
 4. SW3 -> J4
 5. SW4 -> J3
 6. SW5 -> H3
 7. SW6 -> L3
 8. SW7 -> K3
2. Master board clock -> N14
3. Slave board clock -> N14
4. Slave board LEDs:
 1. LED0 -> C11
 2. LED1 -> C12
 3. LED2 -> C13
 4. LED3 -> D13
 5. LED4 -> D11
 6. LED5 -> E11
 7. LED6 -> E12
 8. LED7 -> E13
5. UART TX of master is connected to UART RX of slave.
6. Common ground must be connected between both boards.
7. Both boards must be powered and programmed separately.

Step-by-Step Guide in Vivado for UART Master

Step 1: Create a New Vivado Project

1. Open Vivado and select Create Project.
2. Choose RTL Project and click Next.
3. Specify the project name, for example, uart_master.
4. Add design sources later and click Next.
5. Add constraints later and click Next.
6. Select the Artix-7 part number XC7A15TFTG256-1.
7. Click Next and then Finish.

Step 2: Add the Design Sources

1. In the Flow Navigator, select Add Sources.
2. Choose Add or Create Design Sources.
3. Add the Verilog files for the master design.

4. The design should include the UART transmitter and the master top module.

Step 3: Add Constraints

1. In the Flow Navigator, select Add Sources -> Add or Create Constraints.
2. Create a new XDC file.
3. Add the pin mapping for:
 1. Clock on N14.
 2. Switches on F4, G4, H4, J4, J3, H3, L3, and K3.
 3. UART TX output pin used in the project.
4. Apply LVCMOS33 to the input and output pins.
5. Save the file.

Step 4: Verify the Master Logic

1. Confirm that the master reads the 8 switches as one 8-bit value.
2. Confirm that the UART transmitter sends that value periodically.
3. Confirm that the baud rate is set to 115200.

Step 5: Run Synthesis and Implementation

1. Click Run Synthesis.
2. After synthesis completes, click Run Implementation.
3. After implementation completes, click Generate Bitstream.

Step 6: Program the Master Board

1. Open Hardware Manager.
2. Connect the board through JTAG.
3. Program the generated bitstream.

Step-by-Step Guide in Vivado for UART Slave

Step 1: Create a New Vivado Project

1. Open Vivado and select Create Project.
2. Choose RTL Project and click Next.
3. Specify the project name, for example, uart_slave.
4. Add design sources later and click Next.
5. Add constraints later and click Next.
6. Select the Artix-7 part number XC7A15TFTG256-1.
7. Click Finish.

Step 2: Add the Design Sources

1. Add the UART receiver source file.
2. Add the slave top module that captures received bytes and drives LEDs.

Step 3: Add Constraints

1. Create a new XDC file for the slave design.
2. Add the clock pin N14.
3. Add the UART RX pin used in the project.
4. Add the LED outputs on C11, C12, C13, D13, D11, E11, E12, and E13.
5. Apply LVCMOS33 to all LED pins.
6. Save the constraints file.

Step 4: Verify the Slave Logic

1. Confirm that the UART receiver detects the start bit.
2. Confirm that it reconstructs the 8-bit data byte.
3. Confirm that the received value is stored and sent to the LEDs.

Step 5: Run Synthesis and Implementation

1. Click Run Synthesis.
2. Click Run Implementation.

3. Generate the bitstream.

Step 6: Program the Slave Board

1. Open Hardware Manager.
2. Connect the board through JTAG.
3. Program the generated bitstream.

Step 7: Verify UART Transfer

1. Set the switches on the master board.
2. Observe the slave LEDs.
3. The LED value should match the switch value in binary.
4. Change the switches again and check that the LEDs update after the next UART transmission.

